stockflow-erp-api

Guide des classes

Une fiche par classe : rôle, champs, relations, règles et exemples

Comment lire ce guide

Chaque classe du diagramme a sa page, construite sur le même modèle :

- **Rôle** : à quoi sert la classe, en langage simple
- **Champs** : chaque attribut, son type et ce qu'il signifie
- **Relations** : avec quelles classes elle est liée, et la stratégie de suppression
- **Règles** : les contraintes et règles de gestion (R1, R2, R4, R6, R7, R9…)
- **Exemple** : un cas concret ou un petit schéma
- **À retenir** : le point qui évite les erreurs

> **Rappel de notation. 1 → 0..*** se lit « un élément d'un côté pour zéro ou plusieurs de l'autre ». ◆→ est une composition : la ligne appartient à sa commande et disparaît avec elle (CASCADE). Toutes les autres clés étrangères sont en PROTECT, sauf AuditLog → User (SET_NULL).

Sommaire

Page	Classe	En une phrase
3	**TimestampedModel**	Mixin qui ajoute created_at et updated_at
4	**User**	Personne qui se connecte, avec un rôle
5	**AuditLog**	Journal de qui a fait quoi
6	**Customer / Supplier**	Clients à qui on vend, fournisseurs à qui on achète
7	**Category**	Rayon qui regroupe les produits
8	**Product**	Article du catalogue, identifié par son SKU
9	**Warehouse**	Lieu physique de stockage
10	**StockLevel**	Quantité actuelle d'un produit dans un entrepôt
11	**StockMovement**	Historique immuable des variations de stock
12	**SalesOrder**	Commande d'un client
13	**SalesOrderLine**	Ligne de produit d'une commande de vente
14	**PurchaseOrder**	Commande passée à un fournisseur
15	**PurchaseOrderLine**	Ligne de produit d'une commande d'achat
16	**Receipt**	Bon de réception d'une livraison
17	**ReceiptLine**	Détail des quantités reçues
18	**Les 4 énumérations**	Role, MovementType, SalesStatus, PurchaseStatus
19	**Vue d'ensemble**	Qui dépend de qui, et le trajet d'une vente et d'un achat

TimestampedModel

core · classe abstraite

Rôle

C'est un **modèle outil** : il ne crée aucune table. Il apporte à toutes les classes qui en héritent les deux colonnes d'horodatage, sans les réécrire à chaque fois. On l'appelle un *mixin*.

Champs

Champ	Type	Signification
`created_at`	DateTime	Date et heure de création de la ligne, remplie automatiquement
`updated_at`	DateTime	Date et heure de la dernière modification, mise à jour automatiquement

Relations

Sens	Classe liée	Champ	Suppression	Signification
hérité par	**Presque toutes les classes métier**	–	—	Customer, Supplier, Category, Product, Warehouse, StockLevel, SalesOrder, PurchaseOrder… (voir ci-dessous pour les exceptions)

Règles

- Chaque modèle qui en hérite gagne **created_at** et **updated_at** : c'est la traçabilité de base.
- **Exceptions** : StockMovement et AuditLog n'ont que **created_at**, car ces lignes ne sont jamais modifiées.
- Il est placé dans l'application **core**, le socle commun du projet.

Exemple

```
class TimestampedModel(models.Model):
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        abstract = True      # pas de table pour cette classe
```

À retenir. Sans ce mixin, il faudrait recopier deux lignes dans chaque modèle. Avec lui, la règle « tout est horodaté » est écrite une seule fois.

User

accounts · acteurs du système

Rôle

Représente une **personne qui se connecte** à l'API. Elle sert à deux choses : décider ce que la personne a le droit de faire (son rôle) et savoir qui a fait quoi (champs *created_by*, *received_by*, *actor*).

Champs

Champ	Type	Signification
`email`	EmailField	Identifiant de connexion. Unique et obligatoire
`role`	CharField(choices)	Profil de l'utilisateur. Défaut : VIEWER (lecture seule en pratique, le plus prudent)
`username`	hérité	Conservé pour rester simple, ou supprimé (décision du jour 4)
`(autres champs)`	AbstractUser	mot de passe, nom, prénom, is_active... fournis par Django

Relations

Sens	Classe liée	Champ	Suppression	Signification
1 → 0..*	**StockMovement**	`created_by`	PROTECT	Qui a créé chaque mouvement de stock
1 → 0..*	**SalesOrder**	`created_by`	PROTECT	Qui a créé la commande de vente
1 → 0..*	**PurchaseOrder**	`created_by`	PROTECT	Qui a créé la commande d'achat
1 → 0..*	**Receipt**	`received_by`	PROTECT	Qui a réceptionné la marchandise
0..1 → 0..*	**AuditLog**	`actor`	SET_NULL	Qui a réalisé l'action journalisée

Règles

- Les 5 rôles : **ADMIN, STOCK_MANAGER, SALES, PURCHASER, VIEWER**. Le détail des droits de chacun sera défini dans l'API (document 06).
- **USERNAME_FIELD = "email"** : on se connecte avec l'email.
- Les clés étrangères vers User sont en PROTECT : on ne supprime pas un utilisateur qui a laissé de l'historique (on le désactive).

Exemple

Amina, rôle **SALES**, confirme la commande SO-000012. Le champ *created_by* de la commande pointe vers elle, et une ligne d'audit *sales_order.confirmed* est écrite avec Amina comme acteur.

> **À retenir. À définir dès la première migration.** Changer de modèle utilisateur après coup est très pénible, car toutes les clés étrangères du projet pointent vers lui.

AuditLog

core · journal d'audit

Rôle

Le **journal des actions importantes** : qui a fait quoi, quand, sur quel objet. Il répond à la question « que s'est-il passé sur cette commande ? » des semaines plus tard. Il n'est **jamais modifié**.

Champs

Champ	Type	Signification
`actor`	FK User	Qui a fait l'action. SET_NULL : l'audit survit à la suppression de l'utilisateur
`action`	Char	Nom de l'événement, par exemple sales_order.confirmed
`object_type`	Char	Type de l'objet concerné (une commande de vente, une réception…)
`object_id`	Entier	Identifiant de l'objet concerné
`payload`	JSONField	Détails libres de l'événement (anciennes et nouvelles valeurs, quantités…)
`created_at`	DateTime	Moment de l'action

Relations

Sens	Classe liée	Champ	Suppression	Signification
0..1 → 0..*	**User**	`actor`	SET_NULL	L'acteur peut disparaître, la trace reste

Règles

- Append-only : on ajoute des lignes, on n'en modifie ni n'en supprime jamais.
- L'objet concerné est désigné par un **type + un identifiant**, pas par une clé étrangère : un seul journal peut parler de commandes, de réceptions, etc.
- Ne pas confondre avec StockMovement : le mouvement trace les **quantités**, l'audit trace les **actions des personnes**.

Exemple

Après la confirmation de SO-000012, une ligne est écrite : acteur = Amina, action = *sales_order.confirmed*, object_type = commande de vente, object_id = 12, payload = les quantités réservées.

> **À retenir.** Le choix **SET_NULL** est volontaire : si on utilisait PROTECT, on ne pourrait plus supprimer un utilisateur ; avec CASCADE, on perdrait l'audit.

Customer et Supplier

partners · les partenaires

Rôle

Deux classes jumelles (mêmes champs). **Customer** est une entreprise ou une personne à qui on vend. **Supplier** est celle à qui on achète. Elles sont séparées car leurs flux et leurs règles diffèrent.

Champs

Champ	Type	Signification
`name`	Char(200)	Nom du client ou du fournisseur. Obligatoire
`email`	Email	Facultatif ; unique si renseigné (index unique partiel)
`phone`	Char	Téléphone, facultatif
`address`	Text	Adresse, facultative
`is_active`	Bool	Défaut True. Désactiver remplace supprimer

Relations

Sens	Classe liée	Champ	Suppression	Signification
1 → 0..*	**SalesOrder (depuis Customer)**	`customer`	PROTECT	Un client passe plusieurs commandes
1 → 0..*	**PurchaseOrder (depuis Supplier)**	`supplier`	PROTECT	Un fournisseur reçoit plusieurs commandes

Règles

- **Index unique partiel** sur l'email : deux partenaires ne peuvent pas avoir le même email, mais plusieurs peuvent n'en avoir aucun.
- Une commande de vente exige un **client actif**.
- PROTECT : impossible de supprimer un partenaire qui a des commandes. On le désactive.

Exemple

Le client « Atlas Matériaux » a 14 commandes. On ne peut plus le supprimer ; si on arrête de travailler avec lui, on passe *is_active* à False : il disparaît des nouvelles commandes, mais son historique reste intact.

> **À retenir.** Un index unique classique refuserait deux partenaires sans email (deux valeurs vides identiques). L'index **partiel** ne s'applique qu'aux emails renseignés.

Category

catalog · classement des produits

Rôle

Un **rayon** du catalogue : elle regroupe les produits (par exemple « Visserie », « Outillage »). Tout produit appartient à exactement une catégorie.

Champs

Champ	Type	Signification
`name`	Char(100)	Nom de la catégorie. Unique, sans tenir compte de la casse

Relations

Sens	Classe liée	Champ	Suppression	Signification
1 → 0..*	**Product**	`category`	PROTECT	Une catégorie classe plusieurs produits

Règles

- Unicité insensible à la casse : « Outillage » et « outillage » sont considérés comme le même nom (même technique que pour le SKU).
- PROTECT : on ne supprime pas une catégorie qui contient des produits.

Exemple

Vouloir supprimer la catégorie « Visserie » alors que 40 produits y sont rattachés est refusé par la base. Il faut d'abord déplacer les produits vers une autre catégorie.

> **À retenir.** C'est la classe la plus simple du modèle : un seul champ. Elle sert de filtre et de regroupement dans l'API.

Product

catalog · l'article vendu et acheté

Rôle

Le **catalogue** : un article identifié par son SKU, avec son prix catalogue. C'est la classe centrale, car stock, ventes et achats pointent tous vers elle.

Champs

Champ	Type	Signification
`sku`	Char(40)	Code article. Unique sur Lower(sku) : ABC-1 et abc-1 sont le même SKU (R10)
`name`	Char(200)	Désignation, obligatoire
`description`	Text	Facultative
`category`	FK Category	Obligatoire, PROTECT
`unit_price`	Decimal(12,2)	Prix catalogue actuel, CHECK ≥ 0
`is_active`	Bool	Défaut True. Désactiver retire le produit des nouvelles opérations

Relations

Sens	Classe liée	Champ	Suppression	Signification
0..* ← 1	**Category**	`category`	PROTECT	Le produit appartient à une catégorie
1 → 0..*	**StockLevel**	`product`	PROTECT	Stocké dans plusieurs entrepôts
1 → 0..*	**StockMovement**	`product`	PROTECT	Tracé par ses mouvements
1 → 0..*	**SalesOrderLine**	`product`	PROTECT	Vendu sur plusieurs lignes
1 → 0..*	**PurchaseOrderLine**	`product`	PROTECT	Acheté sur plusieurs lignes

Règles

- **R10** : SKU unique sans tenir compte de la casse.
- Prix ≥ 0 garanti par une contrainte CHECK en base.
- Le prix catalogue peut changer ; le prix d'une commande déjà confirmée, lui, est figé sur sa ligne (R6).

Exemple

```
class Meta:
    constraints = [
        models.UniqueConstraint(Lower("sku"), name="uniq_product_sku_ci"),
        models.CheckConstraint(condition=models.Q(unit_price__gte=0),
                               name="product_price_gte_0"),
    ]    # Django 5.1+ : paramètre condition (check est déprécié)
```

À retenir. Un produit qui a un historique ne se supprime pas (PROTECT partout) : on le désactive avec *is_active*.

Warehouse

inventory · lieu de stockage

Rôle

Un **entrepôt** : l'endroit physique où le stock est conservé. Le même produit peut être présent dans plusieurs entrepôts avec des quantités différentes.

Champs

Champ	Type	Signification
`code`	Char(20)	Code court de l'entrepôt, unique (ex. CASA-01)
`name`	Char(100)	Nom, obligatoire
`is_active`	Bool	Défaut True

Relations

Sens	Classe liée	Champ	Suppression	Signification
1 → 0..*	**StockLevel**	`warehouse`	PROTECT	Les quantités présentes dans l'entrepôt
1 → 0..*	**StockMovement**	`warehouse`	PROTECT	Les mouvements qui s'y produisent
1 → 0..*	**SalesOrder**	`warehouse`	PROTECT	Entrepôt d'où part la commande de vente
1 → 0..*	**PurchaseOrder**	`warehouse`	PROTECT	Entrepôt de destination d'un achat

Règles

- Le code est unique : c'est l'identifiant « lisible » de l'entrepôt.
- PROTECT : un entrepôt qui a des commandes ou du stock ne se supprime pas, on le désactive.

Exemple

La commande SO-000012 part de l'entrepôt CASA-01 : la confirmation réserve le stock **de cet entrepôt** uniquement, pas celui de MRK-01.

> **À retenir.** Le stock n'existe jamais « en général » : il existe toujours pour un couple (produit, entrepôt).

StockLevel

inventory · l'état courant du stock

Rôle

Une ligne = **la quantité d'un produit dans un entrepôt**, à l'instant présent. C'est « la photo » du stock, celle qu'on interroge pour savoir ce qui est disponible.

Champs

Champ	Type	Signification
`product`	FK Product	PROTECT
`warehouse`	FK Warehouse	PROTECT
`on_hand`	Decimal(12,3)	Quantité physiquement présente. CHECK ≥ 0 (R1)
`reserved`	Decimal(12,3)	Quantité promise à des commandes confirmées. CHECK ≥ 0 et ≤ on_hand (R1, R2)
`available()`	propriété	on_hand − reserved. Calculée, jamais stockée

Relations

Sens	Classe liée	Champ	Suppression	Signification
0..* ← 1	**Product**	`product`	PROTECT	Le produit concerné
0..* ← 1	**Warehouse**	`warehouse`	PROTECT	L'entrepôt concerné

Règles

- **Unique(product, warehouse)** : une seule ligne par couple.
- **R1** : on_hand ≥ 0 et reserved ≥ 0. **R2** : reserved ≤ on_hand, donc on ne promet jamais plus que ce qu'on a.
- Modifié uniquement par les services, dans une transaction, avec *select_for_update()*.
- Sa valeur doit toujours égaler la somme des deltas de StockMovement (invariant testé).

Exemple

Situation	on_hand	reserved	available
10 en stock, rien de promis	10	0	10
Commande confirmée de 4	10	4	6
Commande expédiée	6	0	6

À retenir. available n'est pas une colonne. La stocker créerait une donnée en double qui pourrait se désynchroniser. Les trois contraintes CHECK sont le filet de sécurité si un bug contourne le code.

StockMovement

inventory · l'historique (append-only)

Rôle

Le **livre de comptes du stock** : chaque variation de quantité écrit une ligne. Là où StockLevel dit « combien », StockMovement dit « pourquoi, quand et par qui ». On ne le modifie jamais.

Champs

Champ	Type	Signification
`product, warehouse`	FK PROTECT	Où et sur quel article le mouvement a lieu
`movement_type`	Char	RECEIPT, SHIPMENT, RESERVE, RELEASE, ADJUSTMENT
`on_hand_delta`	Decimal	Variation de on_hand, positive ou négative
`reserved_delta`	Decimal	Variation de reserved, positive ou négative
`reference`	Char	Pièce d'origine en texte (SO-000012, PO-000004). Pas de FK vers sales
`reason`	Text	Obligatoire pour ADJUSTMENT
`created_by`	FK User	PROTECT : qui a provoqué le mouvement
`created_at`	DateTime	Automatique

Relations

Sens	Classe liée	Champ	Suppression	Signification
0..* ← 1	**Product / Warehouse**	`product, warehouse`	PROTECT	Le couple concerné
0..* ← 1	**User**	`created_by`	PROTECT	L'auteur du mouvement

Règles

- **Immuable (R9)** : l'API n'expose que GET et save() refuse de modifier une ligne existante.
- **Invariants** : Σ on_hand_delta = StockLevel.on_hand et Σ reserved_delta = StockLevel.reserved, pour chaque couple.
- **reference est un texte** : l'application inventory ne dépend pas de sales ni de purchasing.

Exemple

Type	on_hand_delta	reserved_delta	Lecture logique
RECEIPT	+5	0	Réception d'un achat
RESERVE	0	+4	Confirmation d'une vente
SHIPMENT	−4	−4	Expédition
RELEASE	0	−4	Annulation après confirmation
ADJUSTMENT	±	0 ou ±	Correction manuelle, avec motif

À retenir. Pour corriger une erreur, on n'efface pas un mouvement : on en ajoute un nouveau (ADJUSTMENT, avec son motif). Le tableau des effets est la lecture logique des types ; vos règles exactes priment.

SalesOrder

sales · la commande de vente

Rôle

Une **commande passée par un client**. Elle commence en brouillon, puis on la confirme (le stock est réservé), on l'expédie, ou on l'annule. Elle regroupe des lignes.

Champs

Champ	Type	Signification
`number`	Char	Unique, ex. SO-000012, généré par le service
`customer`	FK Customer	PROTECT ; client actif obligatoire
`warehouse`	FK Warehouse	PROTECT ; entrepôt d'expédition
`status`	Char	DRAFT, CONFIRMED, SHIPPED, CANCELLED
`notes`	Text	Facultatif
`confirmed_at`, `shipped_at`, `cancelled_at`	DateTime null	Dates des transitions, vides tant qu'elles n'ont pas eu lieu
`created_by`	FK User	PROTECT
`total()`	propriété	Somme des line_total des lignes. Pas de colonne stockée au départ

Relations

Sens	Classe liée	Champ	Suppression	Signification
0..* ← 1	**Customer**	`customer`	PROTECT	Qui achète
0..* ← 1	**Warehouse**	`warehouse`	PROTECT	D'où ça part
1 ◆→ 1..*	**SalesOrderLine**	`order`	CASCADE	Ses lignes de produits

Règles

- **R4** : la confirmation est atomique (tout ou rien).
- Confirmer deux fois ne réserve qu'une fois : une commande hors DRAFT lève une erreur de transition.
- Une commande vide (sans ligne) ne peut pas être confirmée.
- **R7** : annuler après confirmation restaure reserved.

Exemple

```
DRAFT --confirmer--> CONFIRMED --expédier--> SHIPPED
  |                      |
  +-----annuler----------+--annuler--> CANCELLED
```

> **À retenir.** C'est le service *confirm_sales_order* qui fait le travail : verrouille la commande, puis chaque StockLevel dans l'ordre des product_id, réserve, écrit les mouvements et fige les prix.

SalesOrderLine

sales · une ligne de la commande

Rôle

Une **ligne de commande** : un produit et une quantité. Elle n'a aucun sens sans sa commande, d'où le CASCADE.

Champs

Champ	Type	Signification
`order`	FK SalesOrder	CASCADE : la ligne disparaît avec la commande
`product`	FK Product	PROTECT
`quantity`	Decimal(12,3)	CHECK > 0 (R14) : pas de ligne à zéro ni négative
`unit_price`	Decimal(12,2) null	Vide en DRAFT ; figé à la confirmation (R6)
`line_total()`	propriété	quantity × unit_price, calculé

Relations

Sens	Classe liée	Champ	Suppression	Signification
0..* ← 1	**SalesOrder**	`order`	CASCADE	La commande propriétaire
0..* ← 1	**Product**	`product`	PROTECT	L'article vendu

Règles

- **Unique(order, product)** : un produit n'apparaît qu'une fois par commande ; pour plus, on augmente la quantité.
- **R14** : quantité strictement positive.
- **R6** : au moment de la confirmation, le prix catalogue du produit est copié dans unit_price.

Exemple

Brouillon : 3 × vis M8, prix vide. Le catalogue affiche 2,00 à la confirmation : la ligne garde 2,00. Le mois suivant, le catalogue passe à 2,50 : cette commande reste à 2,00 et ses totaux ne changent pas.

> **À retenir.** Figer le prix sur la ligne évite qu'une hausse de tarif réécrive l'historique des ventes.

PurchaseOrder

purchasing · la commande d'achat

Rôle

Une **commande passée à un fournisseur** pour réapprovisionner un entrepôt. Elle peut être reçue en une ou plusieurs livraisons.

Champs

Champ	Type	Signification
`number`	Char	Numéro de type PO-000004
`supplier`	FK Supplier	PROTECT
`warehouse`	FK Warehouse	PROTECT ; entrepôt de destination
`status`	Char	DRAFT, ORDERED, PARTIALLY_RECEIVED, RECEIVED, CANCELLED
`ordered_at`	DateTime	Date d'envoi de la commande
`created_by`	FK User	PROTECT

Relations

Sens	Classe liée	Champ	Suppression	Signification
0..* ← 1	**Supplier**	`supplier`	PROTECT	À qui on commande
0..* ← 1	**Warehouse**	`warehouse`	PROTECT	Où ça sera livré
1 ◆→ 1..*	**PurchaseOrderLine**	`purchase_order`	CASCADE	Ses lignes
1 → 0..*	**Receipt**	`purchase_order`	PROTECT	Ses réceptions

Règles

- Le statut avance avec les réceptions : **ORDERED** (rien reçu), **PARTIALLY_RECEIVED** (réception partielle), **RECEIVED** (tout reçu). Cette progression est la lecture logique des quantités reçues.
- Une réception ne peut pas dépasser ce qui a été commandé (R12, sur les lignes).

Exemple

```
DRAFT -> ORDERED -> PARTIALLY_RECEIVED -> RECEIVED
   \        \
    +--------+---------> CANCELLED
```

> **À retenir.** La commande d'achat n'a pas de total stocké ici : le coût vit sur chaque ligne (unit_cost).

PurchaseOrderLine

purchasing · une ligne d'achat

Rôle

Une **ligne d'achat** : quel produit, combien on en a commandé, combien est déjà arrivé, et à quel coût.

Champs

Champ	Type	Signification
`purchase_order`	FK PurchaseOrder	CASCADE
`product`	FK Product	PROTECT
`quantity_ordered`	Decimal	Quantité commandée, > 0
`quantity_received`	Decimal	Quantité déjà reçue, défaut 0
`unit_cost`	Decimal(12,2)	Coût d'achat unitaire, ≥ 0

Relations

Sens	Classe liée	Champ	Suppression	Signification
0..* ← 1	**PurchaseOrder**	`purchase_order`	CASCADE	La commande propriétaire
0..* ← 1	**Product**	`product`	PROTECT	L'article acheté
1 → 0..*	**ReceiptLine**	`po_line`	PROTECT	Les réceptions de cette ligne

Règles

- **R12** : 0 ≤ quantity_received ≤ quantity_ordered, garanti par des CHECK.
- **Unique(purchase_order, product)** : un produit par ligne et par commande.
- **unit_cost** est le prix d'achat ; ne pas le confondre avec le unit_price (prix de vente) du produit.

Exemple

Étape	ordered	received	Résultat
Commande de 100 vis	100	0	ORDERED
1re livraison de 60	100	60	PARTIALLY_RECEIVED
2e livraison de 40	100	100	RECEIVED
Tentative de livrer 10 de plus	100	110	Refusé : 110 > 100

À retenir. Le CHECK en base garantit qu'on ne reçoit jamais plus que commandé, même si le code a un bug.

Receipt

purchasing · une livraison reçue

Rôle

Un **bon de réception** : l'événement « de la marchandise est arrivée » pour une commande d'achat. Une commande peut avoir plusieurs réceptions (livraisons partielles).

Champs

Champ	Type	Signification
`purchase_order`	FK PurchaseOrder	PROTECT
`idempotency_key`	Char	Clé (UUID) envoyée par le client pour éviter les doublons
`received_by`	FK User	Qui a réceptionné
`created_at`	DateTime	Moment de la réception

Relations

Sens	Classe liée	Champ	Suppression	Signification
0..* ← 1	**PurchaseOrder**	`purchase_order`	PROTECT	La commande réceptionnée
1 ◆→ 1..*	**ReceiptLine**	`receipt`	CASCADE	Ses lignes reçues
0..* ← 1	**User**	`received_by`	PROTECT	Le réceptionnaire

Règles

- **R13** : Unique(purchase_order, idempotency_key).
- Si le client rejoue la même requête, le service renvoie la réception existante au lieu de recréer du stock.

Exemple

Requête	Clé	Résultat
1re envoi	a1b2…	Réception créée, stock +5
Rejeu après coupure réseau	a1b2…	Doublon détecté : réception existante renvoyée, stock inchangé

À retenir. Sans idempotence, une coupure réseau suivie d'un nouvel essai pourrait ajouter deux fois la marchandise au stock.

ReceiptLine

purchasing · le détail d'une réception

Rôle

Une ligne d'un bon de réception : **quelle ligne d'achat** est concernée et **quelle quantité** est arrivée cette fois-ci.

Champs

Champ	Type	Signification
`receipt`	FK Receipt	CASCADE
`po_line`	FK PurchaseOrderLine	PROTECT
`quantity`	Decimal	Quantité reçue dans cette livraison, > 0

Relations

Sens	Classe liée	Champ	Suppression	Signification
0..* ← 1	**Receipt**	`receipt`	CASCADE	Le bon de réception
0..* ← 1	**PurchaseOrderLine**	`po_line`	PROTECT	La ligne d'achat servie

Règles

- quantity > 0.
- Elle pointe vers la **ligne d'achat** (po_line), pas directement vers le produit : on sait ainsi exactement quelle commande est servie.
- Chaque ReceiptLine fait augmenter quantity_received de sa ligne d'achat et on_hand du stock (mouvement RECEIPT).

Exemple

Réception REC-1 : ligne PO-000004 / vis M8, quantité 60. Quantity_received passe de 0 à 60, on_hand de la vis augmente de 60 dans l'entrepôt de destination et un mouvement RECEIPT est écrit.

> **À retenir.** Une réception se découpe en lignes pour pouvoir recevoir **seulement une partie** des produits d'une commande.

Les 4 énumérations

types de valeurs autorisées

Une énumération est une liste fermée de valeurs. Elle est stockée dans un champ *CharField(choices)* : la base accepte du texte, mais l'application refuse toute valeur hors de la liste.

Role (sur User)

Valeur	Sens
ADMIN	Administration du système
STOCK_MANAGER	Gestion du stock et des entrepôts
SALES	Ventes : commandes clients
PURCHASER	Achats : commandes fournisseurs et réceptions
VIEWER	Consultation. Valeur par défaut

MovementType (sur StockMovement)

Valeur	Sens (lecture logique)
RECEIPT	Marchandise reçue d'un fournisseur : on_hand augmente
RESERVE	Stock promis à une commande confirmée : reserved augmente
SHIPMENT	Marchandise expédiée : on_hand et reserved diminuent
RELEASE	Réservation libérée (annulation) : reserved diminue
ADJUSTMENT	Correction manuelle, motif obligatoire

SalesStatus (sur SalesOrder)

Valeur	Sens
DRAFT	Brouillon, modifiable, rien n'est réservé
CONFIRMED	Confirmée : stock réservé, prix figés
SHIPPED	Expédiée : le stock est sorti
CANCELLED	Annulée

PurchaseStatus (sur PurchaseOrder)

Valeur	Sens
DRAFT	En préparation
ORDERED	Envoyée au fournisseur
PARTIALLY_RECEIVED	Réceptionnée en partie
RECEIVED	Entièrement reçue
CANCELLED	Annulée

Vue d'ensemble

qui dépend de qui

Les dépendances par bloc

Bloc	Classes	Dépend de
Socle	TimestampedModel, AuditLog	User (AuditLog)
Comptes	User	rien (tout le monde dépend de lui)
Partenaires	Customer, Supplier	rien
Catalogue	Category, Product	Product dépend de Category
Inventaire	Warehouse, StockLevel, StockMovement	Product, Warehouse, User
Ventes	SalesOrder, SalesOrderLine	Customer, Warehouse, Product, User
Achats	PurchaseOrder, PurchaseOrderLine, Receipt, ReceiptLine	Supplier, Warehouse, Product, User

Le trajet d'une vente

```
1. Créer SalesOrder (DRAFT) + SalesOrderLine          [Customer, Warehouse, Product]
2. Confirmer :  verrou  -> StockLevel.reserved +qte
                        -> StockMovement RESERVE
                        -> SalesOrderLine.unit_price figé
                        -> AuditLog sales_order.confirmed
3. Expédier  :  StockLevel.on_hand -qte, reserved -qte -> StockMovement SHIPMENT
   ou Annuler :  StockLevel.reserved -qte              -> StockMovement RELEASE
```

Le trajet d'un achat

```
1. Créer PurchaseOrder (DRAFT) + PurchaseOrderLine    [Supplier, Warehouse, Product]
2. Passer en ORDERED
3. Réceptionner : Receipt (clé d'idempotence) + ReceiptLine
                  -> PurchaseOrderLine.quantity_received +qte
                  -> StockLevel.on_hand +qte -> StockMovement RECEIPT
                  -> statut PARTIALLY_RECEIVED ou RECEIVED
```

Les trois protections qui tiennent l'ensemble : transaction atomique (tout ou rien), verrou de ligne *select_for_update()* (pas deux réservations en même temps) et contraintes CHECK en base (dernier filet). Les mouvements en ajout seul permettent de vérifier à tout moment que l'historique égale l'état courant.